Sequential#

https://pytorch.org/docs/stable/generated/torch.nn.modules.container.Sequent

Description:

A sequential container. Modules will be added to it in the order they are passed in the constructor. Alternatively, an OrderedDict of modules can be passed in. The forward() method of Sequential accepts any input and forwards it to the first module it contains. It then “chains” outputs to inputs sequentially for each subsequent module, finally returning the output of the last module. The value a Sequential provides over manually calling a sequence of modules is that it allows treating the whole container as a single module, such that performing a transformation on the Sequential applies to each of the modules it stores (which are each a registered submodule of the Sequential). What’s the difference between a Sequential and a torch.nn.ModuleList? A ModuleList is exactly what it sounds like–a list for storing Module s! On the other hand, the layers in a Sequential are connected in a cascading way.

Function Signature

```
class torch.nn.modules.container.Sequential(*args: Module)
[source]#

class torch.nn.modules.container.Sequential(arg:
OrderedDict[str, Module])

append(module)[source]#

Parameters

Return type
```

Returns:

Module

Code Examples

```
# Using Sequential to create a small model. When `model` is run,
# input will first be passed to `Conv2d(1,20,5)`. The output of
# `Conv2d(1,20,5)` will be used as the input to the first
# `ReLU`; the output of the first `ReLU` will become the input
# for `Conv2d(20,64,5)`. Finally, the output of
# `Conv2d(20,64,5)` will be used as input to the second `ReLU`
model = nn.Sequential(
    nn.Conv2d(1, 20, 5), nn.ReLU(), nn.Conv2d(20, 64, 5),
nn.ReLU()
)

# Using Sequential with OrderedDict. This is functionally the
# same as the above code
model = nn.Sequential(
    OrderedDict(
        [
            ("conv1", nn.Conv2d(1, 20, 5)),
            ("relu1", nn.ReLU()),
            ("conv2", nn.Conv2d(20, 64, 5)),
            ("relu2", nn.ReLU()),
        ]
    )
)
```

```
>>> import torch.nn as nn
>>> n = nn.Sequential(nn.Linear(1, 2), nn.Linear(2, 3))
>>> n.append(nn.Linear(3, 4))
Sequential(
    (0): Linear(in_features=1, out_features=2, bias=True)
    (1): Linear(in_features=2, out_features=3, bias=True)
    (2): Linear(in_features=3, out_features=4, bias=True)
)
```

```
>>> import torch.nn as nn
>>> n = nn.Sequential(nn.Linear(1, 2), nn.Linear(2, 3))
>>> other = nn.Sequential(nn.Linear(3, 4), nn.Linear(4, 5))
>>> n.extend(other) # or `n + other`
Sequential(
    (0): Linear(in_features=1, out_features=2, bias=True)
    (1): Linear(in_features=2, out_features=3, bias=True)
    (2): Linear(in_features=3, out_features=4, bias=True)
    (3): Linear(in_features=4, out_features=5, bias=True)
)
```

```
>>> import torch.nn as nn
>>> n = nn.Sequential(nn.Linear(1, 2), nn.Linear(2, 3))
>>> n.insert(0, nn.Linear(3, 4))
Sequential(
    (0): Linear(in_features=3, out_features=4, bias=True)
    (1): Linear(in_features=1, out_features=2, bias=True)
    (2): Linear(in_features=2, out_features=3, bias=True)
)
```

Detailed Documentation

Documentation

A sequential container.

Modules will be added to it in the order they are passed in the constructor. Alternatively, an OrderedDict of modules can be passed in. The forward() method of Sequential accepts any input and forwards it to the first module it contains. It then

“chains” outputs to inputs sequentially for each subsequent module, finally returning the output of the last module.

The value a Sequential provides over manually calling a sequence of modules is that it allows treating the whole container as a single module, such that performing a transformation on the Sequential applies to each of the modules it stores (which are each a registered submodule of the Sequential).

What’s the difference between a Sequential and a torch.nn.ModuleList? A ModuleList is exactly what it sounds like–a list for storing Module s! On the other hand, the layers in a Sequential are connected in a cascading way.

Append a given module to the end.

module (nn.Module) – module to append

Extends the current Sequential container with layers from another Sequential container.

sequential (Sequential) – A Sequential container whose layers will be added to the current container.

Runs the forward pass.

Inserts a module into the Sequential container at the specified index.

index (int) – The index to insert the module.

module (Module) – The module to be inserted.